

Using Python to Solve Linear Programs

Lecture 1 & 2

Danko Turcic

October 5, 2026

Agenda

- Python Basics
- Google Colab Environment
- Introduction to PuLP
- The 4-Step PuLP Framework

Why Python for Business LPs?

- **Beyond Excel:** Handles much larger datasets.
- **Scalability:** Easy to scale from 10 variables to 10,000+.
- **Automation:** Integrates with databases and APIs.
- **Industry Standard:** Most OR and data science teams use Python.

The Environment: Google Colab

- Cloud-based Jupyter Notebook environment.
- **Zero Setup:** Runs entirely in the browser.
- Free access to computing resources.
- Easy sharing and collaboration.

Navigating Colab

- **Code Cells:** Where you write and execute Python.
- **Text Cells:** Where you write documentation (Markdown).
- **Execution:** Press `Shift + Enter` to run a cell.
- Variables are saved in memory after a cell runs.

The `print()` function

Used to output text to the screen.

```
print("Hello, Optimization World!")
```

- Text must be enclosed in quotes.
- Useful for debugging and viewing results.

Variables

- A variable is a named container for storing information in memory.
- No need to declare types explicitly in Python.
- Examples:
 - `capacity = 100`
 - `product_name = "Chair"`

Data Types

- **Integers:** Whole numbers (e.g., 10, -5).
- **Floats:** Decimal numbers (e.g., 3.14, 0.5).
- **Strings:** Text (e.g., "Optimize").
- *Why precision matters:* Floats are crucial for continuous LP variables.

Lists (Arrays)

- Grouping ordered data together.
- Created using square brackets `[]`.

```
products = ['A', 'B', 'C']  
costs = [10.5, 8.0, 12.2]
```

List Operations

- **Accessing Elements:** Zero-indexed!

```
first_product = products[0] # Returns 'A'
```

- **Adding Items:**

```
products.append('D')
```

Dictionaries (Hashmaps) - Concept

- Store data in **Key-Value** pairs.
- Like a real dictionary: look up a word (key) to find the definition (value).
- Perfect for mapping sets to parameters.

Dictionaries - Practice

- Created using curly braces {}.

```
profits = {'A': 20, 'B': 30, 'C': 40}  
  
# Accessing a value  
profit_b = profits['B'] # Returns 30
```

Why Objects Matter in LPs

Python Object

- Lists (`['A', 'B']`)
- Dictionaries

Math Concept

- Sets (Indices $i \in I$)
- Parameters (c_i, a_{ij})

What is a Python Library?

- A collection of pre-written code.
- Prevents reinventing the wheel.
- You "import" libraries to use their functions.
- Examples: Pandas (data), Matplotlib (graphs).

- An open-source linear programming library.
- Allows mathematical formulation in Python.
- Interfaces with solvers (CBC, GLPK, Gurobi, CPLEX).
- Intuitive syntax for constraints and objectives.

Installing PuLP

- PuLP is not installed by default in Colab.
- Use the package installer pip.

```
!pip install pulp
```

- The exclamation mark (!) runs terminal commands in Colab.

Importing PuLP

- Once installed, it must be imported into the script.

```
import pulp as pl
```

- as pl creates a short alias.
- Alternatively: `from pulp import *` (not recommended).

The 4-Step Framework

- 1 **Initialize:** Create the model object.
- 2 **Variables:** Define decision variables.
- 3 **Objective:** Set the objective function.
- 4 **Constraints:** Add equations/inequalities.

Step 1: Initialize the Model

- Creates the core problem object.

```
# For Maximization
model = pl.LpProblem("My_Problem", pl.LpMaximize)

# For Minimization
model = pl.LpProblem("My_Problem", pl.LpMinimize)
```

Step 2: Define Decision Variables (Simple)

- Use `model.add_variable()`.
- Specify name, bounds, and category.

```
x = model.add_variable("x", lowBound=0, cat='Continuous')  
y = model.add_variable("y", lowBound=0, cat='Integer')
```

Step 3: Define Objective Function

- Use the += operator to add expressions to the model.

```
# Maximize 3x + 2y  
model += 3*x + 2*y, "Objective_Function"
```

Step 4: Define Constraints

- Also use the += operator.
- Must include <=, >=, or ==.

```
# Constraints
model += 2*x + y <= 100, "Resource_1"
model += x + 2*y <= 80, "Resource_2"
```

- Call `solve()` on the model.
- Check the status (Optimal, Infeasible, Unbounded).

```
model.solve()

print("Status:", pl.LpStatus[model.status])
```

Readability Counts

"Code is read much more often than it is written." - PEP 8

- Write code for humans, not just for the computer.
- Clear logic prevents modeling errors.

Best Practice 1: Meaningful Names

- Avoid single-letter variables in business applications.

Bad:

```
x = model.add_variable("x")
```

Good:

```
chairs_produced = model.add_variable("Chairs")
```

Best Practice 2: Using Comments

- Use # to explain *why*, not *what*.

Bad:

```
# Add constraint  
model += x <= 50
```

Good:

```
# Maximum warehouse capacity based on Q3 audit  
model += chairs_produced <= 50
```

Best Practice 3: Start Simple

- **Iterative Development:** Don't build the final complex model immediately.
- Hardcode simple values first to ensure the logic works.
- Gradually replace hardcoded values with lists, dictionaries, and dynamic sets.

The AI Trap

- **Temptation:** Asking ChatGPT to write your LP from scratch.
- **Danger:** AI often invents constraints or hallucinates math logic.
- If you don't know the fundamentals, you cannot verify if the AI's output is correct.

- **Proper Usage:** Use AI *after* grasping fundamentals.
- Good for:
 - Debugging syntax errors.
 - Suggesting performance optimizations.
 - Explaining specific error codes.

Wrap-up & Q&A (Lecture 1)

- **Summary:** Python basics, Colab, PuLP 4-step framework.
- **Next Lecture:** Applying this to real business cases and scaling up with data structures.
- Questions?

Python Basics: Introduction

- Python is a high-level, interpreted programming language.
- Known for its readability and concise syntax.
- Widely used in data science, optimization, and automation.

Python Basics: Variables & Types

- Variables are created when assigned a value: `x = 5`
- Common data types include:
 - Integers (`int`): 5, -3
 - Floats (`float`): 3.14, -0.001
 - Strings (`str`): "Hello"
 - Booleans (`bool`): True, False

Python Basics: Basic Operations

- Arithmetic operations: $+$, $-$, $*$, $/$
- Exponentiation: $**$ (e.g., $2**3 = 8$)
- Modulo: $%$ (e.g., $10\%3 = 1$)
- Comparison operators: $==$, $!=$, $>$, $<$

Python Basics: Control Flow - If Statements

- Used for conditional execution.
- Indentation is mandatory in Python.

Example

```
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

Python Basics: Control Flow - Loops

- for loops iterate over a sequence.
- while loops repeat as long as a condition is true.

For Loop Example

```
for i in range(5):  
    print(i)      # Prints 0 to 4
```

Lists: Introduction

- Lists are ordered, mutable collections of items.
- Created using square brackets: `my_list = [1, 2, 3]`
- Can contain mixed data types: `[1, "apple", True]`

Lists: Indexing & Slicing

- Python uses 0-based indexing.
- Access items: `my_list[0]` (first item).
- Negative indexing: `my_list[-1]` (last item).
- Slicing: `my_list[1:3]` (elements at index 1 and 2).

Lists: Common Methods

- `append(x)`: Adds an item to the end.
- `remove(x)`: Removes the first occurrence of item `x`.
- `pop()`: Removes and returns the last item.
- `len(my_list)`: Returns the number of items.

Lists: Iterating over Lists

- Easily iterate using a for loop.

Example

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Lists: List Comprehensions

- A concise way to create lists.
- Syntax: `[expression for item in iterable if condition]`

Example

```
squares = [x**2 for x in range(5)]  
# Result: [0, 1, 4, 9, 16]
```

Dictionaries: Introduction

- Dictionaries are unordered collections of key-value pairs.
- Created using curly braces: `my_dict = {"a": 1, "b": 2}`
- Keys must be unique and immutable (e.g., strings, numbers).

Dictionaries: Accessing Values

- Access values using their corresponding keys.
- Example: `my_dict["a"]` returns 1.
- Using `get()` avoids `KeyError`: `my_dict.get("c", 0)` returns 0 if "c" is not found.

Dictionaries: Modifying Content

- Add or update an entry: `my_dict["c"] = 3`
- Delete an entry: `del my_dict["a"]`
- Check for a key: `"b" in my_dict` returns `True`.

Dictionaries: Common Methods

- `keys()`: Returns a view of all keys.
- `values()`: Returns a view of all values.
- `items()`: Returns a view of all (key, value) tuples.

Dictionaries vs Lists

- Use **Lists** for ordered data where index/position matters.
- Use **Dictionaries** for mapping relationships or quick lookups by a unique key.
- Both are extensively used when preparing data for optimization models.

PuLP Setup: What is PuLP?

- PuLP is an open-source Linear Programming (LP) modeler written in Python.
- Allows you to define decision variables, constraints, and objective functions mathematically.
- Interfaces with solvers like CBC, GLPK, CPLEX, and Gurobi.

PuLP Setup: Installation

- PuLP can be installed via Python's package manager, pip.
- Standard installation includes the default CBC solver.

Terminal Command

```
pip install pulp
```

PuLP Setup: Importing the Library

- Once installed, import it into your Python script.
- Common convention is to import everything or specific classes.

Python Code

```
from pulp import *
```

PuLP Setup: Initializing a Model

- Create an LP problem object using `LpProblem`.
- You must specify the problem name and the objective sense (Maximize or Minimize).

Example

```
prob = LpProblem("My_Optimization_Model", LpMaximize)
```

PuLP Setup: Model Structure

- **Variables:** Created with `model.add_variable()`.
- **Objective Function:** Added to the problem with `+=`.
- **Constraints:** Inequalities or equalities added with `+=`.
- **Solve:** Triggered by calling `prob.solve()`.

Using Python to Solve Linear Programs

Part 2

Agenda

- Review of PuLP basics
- Solving a simple business case (Artisan Furniture)
- Scaling with Lists and Dictionaries
- Advanced Constraints and Iteration
- Troubleshooting models

Recap: 4 Steps & Data Types

PuLP Steps:

- 1 Initialize Model
- 2 Variables
- 3 Objective
- 4 Constraints

Python Types:

- Integers & Floats
- Lists (`[]`)
- Dictionaries (`{}`)

Case Study 1: Artisan Furniture Co.

- **Products:** Tables and Chairs.
- **Profit:** Table (\$40), Chair (\$30).
- **Wood constraint:** 300 units available. Table uses 3, Chair uses 2.
- **Labor constraint:** 110 hours available. Table uses 1, Chair uses 1.

Formulation on Paper

- **Variables:** $T, C \geq 0$
- **Objective:** Maximize $Z = 40T + 30C$
- **Constraints:**
 - $3T + 2C \leq 300$ (Wood)
 - $1T + 1C \leq 110$ (Labor)

Translating to Code (Step 1 & 2)

```
import pulp as pl

# 1. Initialize
model = pl.LpProblem("Artisan", pl.LpMaximize)

# 2. Variables
T = model.add_variable("Tables", lowBound=0)
C = model.add_variable("Chairs", lowBound=0)
```

Translating to Code (Step 3 & 4)

```
# 3. Objective
```

```
model += 40*T + 30*C, "Profit"
```

```
# 4. Constraints
```

```
model += 3*T + 2*C <= 300, "Wood"
```

```
model += 1*T + 1*C <= 110, "Labor"
```

Solving and Outputting Results

```
model.solve()

print("Status:", pl.LpStatus[model.status])
print("Tables:", T.varValue)
print("Chairs:", C.varValue)
print("Total Profit:", pl.value(model.objective))
```

- Output tells us exactly how many of each to produce.
- Binding constraints (using 100% capacity) indicate bottlenecks.
- Non-binding constraints indicate excess capacity.

The Problem with Simple Variables

- What if we have 50 products instead of 2?
- Writing 50 `LpVariables` by hand is inefficient and error-prone.
- We need a way to generate variables dynamically.

Enter Data Structures

- We will use Lists to define our **Sets** (Products).
- We will use Dictionaries to define our **Parameters** (Profits, Costs).
- This separates the *Data* from the *Model Structure*.

Defining Sets

- Create a Python List for products.

```
products = ['Table', 'Chair', 'Desk', 'Stool']
```

Defining Parameters

- Create Dictionaries mapped to the List keys.

```
profits = {'Table': 40, 'Chair': 30,  
          'Desk': 50, 'Stool': 15}  
  
wood_used = {'Table': 3, 'Chair': 2,  
            'Desk': 4, 'Stool': 1}
```

Advanced Variables: dicts()

- Generate a dictionary of variables from the list.

```
# Creates a variable for every item in 'products'
prod_vars = pl.LpVariable.dicts("Prod",
                                products,
                                lowBound=0)

# Access table variable: prod_vars['Table']
```

The Power of lpSum

- `pl.lpSum()` is the Python equivalent of mathematical summation (Σ).
- It efficiently adds up elements in a list.

```
# Sums all items in a list comprehension  
pl.lpSum([profits[i] * prod_vars[i] for i in products])
```

Refactoring the Objective

- Using data structures and lpSum.

```
model = pl.LpProblem("Scale", pl.LpMaximize)

# Objective: sum of profit * variable
model += pl.lpSum([profits[i] * prod_vars[i]
                   for i in products])
```

Refactoring Constraints

- Adding constraints cleanly.

```
# Total wood constraint
model += pl.lpSum([wood_used[i] * prod_vars[i]
                   for i in products]) <= 300
```

Case Study 2: The Logistics Problem

- **Goal:** Minimize transportation costs.
- **Sources:** Factories (A, B).
- **Destinations:** Warehouses (X, Y, Z).
- We need a variable for every route (e.g., A to X).

Logistics Setup (Two Indices)

```
factories = ['A', 'B']
warehouses = ['X', 'Y', 'Z']

# Tuple keys for variables (Factory, Warehouse)
routes = [(f, w) for f in factories
           for w in warehouses]

route_vars = pl.LpVariable.dicts("Route",
                                  routes,
                                  lowBound=0)
```

```
# Example constraint: Factory A supply limit
model += pl.lpSum([route_vars[('A', w)]
                   for w in warehouses]) <= 100

# Iterate over factories to create all supply constr.
for f in factories:
    model += pl.lpSum([route_vars[(f, w)]
                       for w in warehouses]) <= supply[f]
```

- Solve as usual.
- Loop through variables to print routes $i \geq 0$.
- Analyzing these routes helps optimize supply chain networks efficiently.

- **Syntax Errors:** Python errors (e.g., missing brackets, `KeyError`).
- **Math Errors:** Code runs, but results make no sense.
- Fix syntax first, then inspect math logic by checking constraint outputs.

- **Status:** Infeasible.
- **Meaning:** Constraints contradict each other. No solution exists.
- **Diagnosis:** Example: $X \geq 10$ and $X \leq 5$. Check resource capacities vs. demand requirements.

Unbounded Models

- **Status:** Unbounded.
- **Meaning:** Objective can go to infinity.
- **Diagnosis:** You forgot a constraint! (e.g., maximizing profit but forgetting material limits).

Code Review: Explicit vs. Scalable

Explicit (Slide 37)

- Easy to read initially.
- Hard to change.
- Terrible for 100+ variables.

Scalable (Slide 47)

- Uses lists and loops.
- Data is separated.
- Solves any size problem.

Excel Solver vs. Python

Feature	Excel	Python
Setup Speed	Fast	Slower
Scalability	Low	High
Automation	Poor	Excellent
Reproducibility	Fair	Excellent

Extracting Results to DataFrames

- PuLP output can be passed into **Pandas**.
- Create a DataFrame of optimal variables.
- Useful for exporting to CSV or visualizing in dashboards.

Summary of the Journey

- Started with basic Python syntax.
- Modeled explicit algebra in PuLP.
- Graduated to lists, dictionaries, and dynamic sets.
- You are now equipped to build scalable algorithms!

Final Advice on Learning

- **Practice:** Write models from scratch.
- **Fundamentals:** Don't skip them for quick fixes.
- **Patience:** Debugging math is normal.

Questions?

Thank you for participating!

Scaling Up PuLP Models

- As LP problems grow, explicitly defining every variable and term becomes impossible.
- PuLP provides powerful functions to scale models dynamically.
- Key tools for scaling:
 - `lpSum` for efficient summations.
 - `LpVariable.dicts` for batch variable creation.

What is lpSum?

A highly optimized PuLP function designed to sum expressions involving `LpVariable` objects.

- Calculates linear expressions significantly faster than Python's built-in `sum()` function.
- Retains the internal PuLP data structures required for passing to the solver.
- The standard way to formulate objective functions and constraints with many terms.

The Power of List Comprehensions

- `lpSum` is most effective when paired with Python list comprehensions.
- Instead of writing:
$$x1 + x2 + x3 + x4 + x5$$
- We can write:
`lpSum([x[i] for i in range(5)])`
- This cleanly separates the *data* (the range or list) from the *model structure*.

Example: lpSum for Objective Functions

Cost Minimization

Suppose we have a dictionary of costs and a dictionary of variables.

```
costs = {'A': 10, 'B': 15, 'C': 20}
# Assume x is a dict of PuLP variables

# Building the objective:
prob += lpSum([costs[i] * x[i] for i in ['A', 'B', 'C']])
```

- This dynamically creates $10x_A + 15x_B + 20x_C$.

Example: lpSum for Constraints

Capacity Constraint

Ensuring total production does not exceed capacity.

```
products = ['P1', 'P2', 'P3']  
capacity = 100
```

```
# Adding constraint to model:
```

```
prob += lpSum([x[p] for p in products]) <= capacity
```

- Adding or removing products only requires changing the products list!

Creating Multiple Variables: `LpVariable.dicts`

- Creating variables one by one (`x1 = LpVariable(...)`) is tedious and error-prone.
- `LpVariable.dicts` creates a dictionary of variables in a single line.
- The dictionary keys correspond to the indices of your mathematical model.

Syntax of LpVariable.dicts

Function Signature

```
LpVariable.dicts(name, indexs,  
                 lowBound=None, upBound=None,  
                 cat='Continuous')
```

- **name**: Prefix for all variable names in the model.
- **indexs**: A list (or iterable) of keys for the dictionary.
- **lowBound**, **upBound**: Apply to **all** variables created.
- **cat**: 'Continuous', 'Integer', or 'Binary'.

Example: 1D Variable Dictionary

Creating Variables for Products

```
products = ['Apples', 'Bananas', 'Cherries']
```

```
produce = LpVariable.dicts("Prod", products,  
                           lowBound=0,  
                           cat='Integer')
```

- Creates: Prod_Apples, Prod_Bananas, Prod_Cherries.
- Access them in Python as produce['Apples'].

Combining dicts and lpSum

- These two tools are meant to be used together.

1. Define Data

```
months = ['Jan', 'Feb', 'Mar']  
demand = {'Jan': 100, 'Feb': 150, 'Mar': 200}
```

2. Create Variables

```
production = LpVariable.dicts("Prod", months, lowBound=0)
```

3. Use lpSum for total production

```
total_prod = lpSum([production[m] for m in months])
```

Modeling 2D Relationships

- Many real-world problems exist in two dimensions:
 - Shipping from i Factories to j Warehouses.
 - Assigning n Workers to m Shifts.
 - Investing in a Assets over t Time periods.
- This requires variables with two indices: $x_{i,j}$.

Creating 2D Index Tuples

- To create a 2D dictionary of variables, we first need a list of all (i, j) combinations.
- A list comprehension makes this easy.

```
factories = ['F1', 'F2']  
warehouses = ['W1', 'W2', 'W3']  
  
routes = [(f, w) for f in factories  
           for w in warehouses]
```

- Result: `[('F1', 'W1'), ('F1', 'W2'), ...]`

Passing Tuples to LpVariable.dicts

```
# Create variables mapped to the tuples
shipping = LpVariable.dicts("Route",
                            routes,
                            lowBound=0,
                            cat='Integer')
```

- The dictionary keys are now tuples.
- We access the variable from F1 to W2 as:
`shipping[('F1', 'W2')]` or `shipping['F1', 'W2']`.

2D Summations: Marginal Constraints

Summing over Columns or Rows

We often need to fix one index and sum over the other.

- Example: Total shipped **out of** Factory F1 to **all** Warehouses must not exceed supply.

```
for f in factories:
    prob += lpSum([shipping[f, w]
                   for w in warehouses]) <= supply[f]
```

2D Summations: The Objective Function

Double Summations

Multiplying 2D costs by 2D variables.

Assuming 'costs' is a dictionary keyed by (f, w) tuples

```
prob += lpSum([costs[f, w] * shipping[f, w]
               for f in factories
               for w in warehouses])
```

- Notice the nested loop inside the list comprehension.
- This neatly constructs $\sum_i \sum_j c_{ij} x_{ij}$.

Summary of Advanced Modeling

- **Data Separation:** Keep indices, costs, and bounds in standard Python lists/dicts.
- **Variable Generation:** Use `LpVariable.dicts` to map variables directly to your data keys.
- **Expression Building:** Use list comprehensions and `lpSum` to construct constraints dynamically.
- This approach ensures your models can scale from 10 variables to 1,000,000 without rewriting code!

Debugging PuLP Models

- Building large models dynamically means you might not see the mathematical equations.
- When the solver returns an unexpected result, how do you fix it?
- Key strategies:
 - Check model status.
 - Inspect the generated `.lp` file.
 - Print variable values and constraint slack.

Checking Model Status

- Always verify the optimization status before using the results!

```
prob.solve()  
print("Status:", LpStatus[prob.status])
```

Common Status Codes

- **Optimal:** The best solution was found.
- **Infeasible:** Constraints contradict each other; no solution exists.
- **Unbounded:** The objective can be improved infinitely.

Printing and Inspecting

- You can print the entire PuLP problem object to the console:
- `print(prob)`
- Iterating over variables to see what the solver decided:

```
for v in prob.variables():  
    if v.varValue > 0: # Only print non-zero values  
        print(v.name, "=", v.varValue)
```

The .lp File: The Source of Truth

- If constraints are wrong, `print(prob)` can be cluttered.
- Export the model to a standard LP formatted text file.

```
prob.writeLP("my_model.lp")
```

- Open `my_model.lp` in any text editor.
- This shows the exact, expanded mathematical equations that PuLP sent to the solver.
- Crucial for debugging list comprehensions!

Dealing with Infeasibility

Why is my model Infeasible?

Usually caused by conflicting constraints or overly restrictive bounds.

- **Strategy 1:** Relax constraints. Comment them out one by one to find the culprit.
- **Strategy 2:** Add "slack" variables with massive penalty costs in the objective to force a solution and see where the violation occurs.
- **Strategy 3:** Double-check $\leq$ vs $=$ signs.